



دانشگاه دامغان

دانشکده فنی و مهندسی

پایان نامه دوره کارشناسی مهندسی کامپیوتر

سایت اشتراک گذاری API با پایتون

سجاد رضاقلی زاده

استاد راهنما:

دکتر زهره کریمی

تیر ۱۴۰۵

بسمه تعالی

تشکر و قدردانی

از استاد راهنما و همه کسانی که در مسیر انجام این پروژه با راهنمایی، پیگیری و دلگرمی کنار من بودند، صمیمانه سپاسگزارم.

چکیده

این گزارش درباره یک پلتفرم فارسی برای اشتراک گذاری، جستجو و مصرف API است؛ سامانه‌ای که تلاش می‌کند تجربه کشف سرویس، خواندن مستندات، مدیریت حساب و انتشار API را در یک مسیر پیوسته کنار هم قرار دهد. در این پروژه، بک‌اند با Django REST Framework و رابط کاربری با React ساخته شده و فضای کلی آن به API Marketplace نزدیک است. کاربر می‌تواند کاتالوگ را ببیند، مستندات را بررسی کند، وارد حساب خود شود، مصرف و اشتراک را دنبال کند، API منتشر کند و از ابزارهایی مانند Caller و Studio کمک بگیرد.

گزارش از دو زاویه به پروژه نگاه می‌کند: تجربه کاربر و ساختار فنی. ابتدا روشن می‌شود کاربر با سامانه چه کاری انجام می‌دهد و سپس توضیح داده می‌شود این رفتارها چگونه میان رابط کاربری، بک‌اند و لایه داده تقسیم شده‌اند. منابع فنی از داکيومنت‌های رسمی مانند Django، Python، Docker، Node.js، React، Django REST Framework، OpenAPI و RapidAPI انتخاب شده‌اند. برای ارزیابی بک‌اند نیز دستور `python manage.py test api` اجرا شد و نتیجه آن در فصل آزمون آمده است.

کلیدواژه‌ها: پلتفرم اشتراک گذاری API، Django REST Framework، React، احراز هویت، OpenAPI

فهرست مطالب

د	فهرست علائم و نشانه‌ها	
ه	فهرست جدول‌ها	
و	فهرست نمودارها، عکس‌ها و نقشه‌ها	
۱	مقدمه	۱
۱	۱-۱ زمینه پروژه	۱
۱	۱-۲ بیان مسئله	۱
۲	۱-۳ اهداف	۲
۲	۱-۴ دامنه و محدودیت‌ها	۲
۲	۱-۵ ذی‌نفعان	۲
۲	۱-۶ سازمان‌دهی گزارش	۲
۳	۱-۷ جمع‌بندی فصل	۳
۴	۲ فناوری‌ها و پیشینه	
۴	۲-۱ Django, Python و Django REST Framework	۴
۴	۲-۲ MongoDB و مدل داده	۴
۵	۲-۳ Vite و TypeScript, React	۵
۵	۲-۴ Node.js و ابزارهای فرانت‌اند	۵
۵	۲-۵ Docker Compose و اجرای محلی	۵
۵	۲-۶ OpenAPI و RapidAPI	۵
۶	۲-۷ جمع‌بندی فصل	۶
۷	۳ تحلیل نیازمندی‌ها	
۷	۳-۱ نقش‌های کاربری	۷
۷	۳-۲ نیازمندی‌های عملکردی و غیرفعال	۷
۹	۳-۳ ورودی‌ها، خروجی‌ها و اعتبارسنجی	۹
۹	۳-۴ امنیت	۹

۳-۵	جمع‌بندی فصل	۱۰
۴	معماری و طراحی	۱۱
۴-۱	نمای کلی	۱۱
۴-۲	نمودار معماری	۱۱
۴-۳	مدل داده	۱۲
۴-۴	مرزهای خارجی	۱۳
۴-۵	جمع‌بندی فصل	۱۳
۵	پیاده‌سازی	۱۴
۵-۱	مسیرهای بک‌اند	۱۴
۵-۲	Repository لایه	۱۴
۵-۳	Serializer و قرارداد پاسخ	۱۵
۵-۴	فرانت‌اند	۱۵
۵-۵	جریان‌های اصلی پیاده‌سازی	۱۵
۵-۶	بخش‌های فنی	۱۶
۵-۷	نمونه کد	۱۷
۵-۸	جمع‌بندی فصل	۱۷
۶	آزمون و ارزیابی	۱۸
۶-۱	راهبرد آزمون	۱۸
۶-۲	نتایج اجرا	۱۸
۶-۳	جدول آزمون‌ها	۱۹
۶-۴	محدودیت‌های آزمون	۲۰
۶-۵	جمع‌بندی فصل	۲۰
۷	نتایج و بحث	۲۱
۷-۱	ویژگی‌های پیاده‌سازی شده	۲۱
۷-۲	نمونه خروجی API	۲۱
۷-۳	نقاط قوت	۲۲
۷-۴	چالش‌ها و بدهی فنی	۲۲
۷-۵	بحث	۲۲
۷-۶	جمع‌بندی فصل	۲۳
۸	نتیجه‌گیری و کارهای آینده	۲۴
۸-۱	جمع‌بندی کار انجام‌شده	۲۴
۸-۲	دستاوردهای اصلی	۲۴
۸-۳	محدودیت‌های فعلی	۲۵

۲۵	کارهای آینده	۸-۴
۲۵	جمع‌بندی فصل	۸-۵

۲۶ منابع

۲۸	الف راهنمای اجرا و شواهد تکمیلی
۲۸	۱-الف اجرای محلی
۲۸	۲-الف اجرای آزمون بک‌اند
۲۸	۳-الف مسیرهای مهم

فهرست علائم و نشانه‌ها

نماد	شرح
API	رابط برنامه‌نویسی کاربردی
DRF	Django REST Framework
SPA	برنامه تک‌صفحه‌ای
RTL	راست‌به‌چپ
ORM	نگاشت شیء-رابطه

فهرست جداول

۳-۱	نیازمندی‌های اصلی پروژه	۷
۴-۱	گروه‌های داده اصلی	۱۲
۵-۱	جریان‌های اصلی در پیاده‌سازی	۱۵
۵-۳	بخش‌های فنی و نقش آن‌ها در پروژه	۱۶
۶-۱	خلاصه آزمون‌های اجراشده	۱۹
الف-۱	مسیرهای مهم برای بررسی دستی سامانه	۲۸

فهرست نمودارها، عکس‌ها و نقشه‌ها

۳-۱	نمای نقش‌ها و جریان‌های اصلی پروژه	۸
۴-۱	معماری سطح بالای پروژه	۱۲

فصل ۱

مقدمه

در شروع گزارش، بهتر است پروژه را از همان جایی ببینیم که کاربر با آن روبه‌رو می‌شود: یک محیط فارسی برای پیدا کردن، بررسی، انتشار و مصرف API. اگر این تصویر روشن باشد، توضیحات فنی فصل‌های بعد هم معنا پیدا می‌کنند؛ چون هر مسیر، جدول و تصمیم معماری در نهایت باید به یک تجربه قابل فهم برای کاربر یا توسعه‌دهنده برسد.

1-1 زمینه پروژه

IranAPI Hub یک پلتفرم فارسی برای معرفی، جستجو، انتشار و مصرف API است. فضای پروژه به بازارچه‌های API نزدیک است؛ جایی که کاربر قبل از استفاده از یک سرویس، می‌تواند آن را بشناسد، مستنداتش را بخواند، پلن‌ها را مقایسه کند و بعد از ورود، مصرف و دسترسی‌های خود را مدیریت کند. این نگاه با مدل رایج API Hub و بازارچه‌های API هم‌خوان است [۱۱]. از سمت فنی، پروژه روی ترکیب Python، Django REST Framework، React، Vite، MongoDB و Docker Compose شکل گرفته است [۱، ۲، ۳، ۴، ۶، ۹، ۸]. این ترکیب برای سامانه‌ای مناسب است که هم به API ساخت‌یافته نیاز دارد و هم به رابط کاربری سریع، فارسی و قابل توسعه.

2-1 بیان مسئله

کاربر چنین سامانه‌ای معمولاً دنبال یک فهرست ساده از سرویس‌ها نیست. او می‌خواهد سریع بفهمد هر API چه کاری انجام می‌دهد، مستندات هر Endpoint چگونه نوشته شده، چه پلن‌هایی دارد و بعد از ورود، وضعیت مصرف، کلیدها، اشتراک و سازمان خود را کجا می‌تواند ببیند. از طرف دیگر، توسعه‌دهنده هم فقط مصرف‌کننده نیست؛ باید بتواند API تازه منتشر کند، یک درخواست نمونه بفرستد و جریان کاری خود را در Studio تنظیم کند.

مسئله اصلی از همین جا شروع می‌شود: تجربه کاربر نباید پخش و تکه‌تکه باشد. وقتی مستندات، پرداخت، احراز هویت، مصرف و ابزار تست هر کدام در فضای جداگانه‌ای قرار بگیرند، کاربر برای رسیدن به نتیجه چند بار مسیر ذهنی خود را عوض می‌کند. این پروژه همین اجزا را در

یک تجربه منسجم جمع می‌کند؛ تجربه‌ای که برای کاربر عمومی قابل فهم باشد و برای توسعه‌دهنده هم کاربرد عملی داشته باشد.

3-1 اهداف

با این نگاه، هدف‌های اصلی پروژه چنین تعریف شده‌اند:

- ارائه کاتالوگ API با جستجو، دسته‌بندی، امتیاز، پلن و مستندات.
- فراهم کردن ثبت‌نام، ورود، خروج و نشست کاربر با مدل رایج احراز هویت وب.
- نمایش Dashboard توسعه‌دهنده شامل پروفایل، API Key، مصرف، سازمان و اشتراک.
- پشتیبانی از انتشار API، اجرای نمونه درخواست در Caller و ساخت جریان در Studio.
- ارائه قرارداد OpenAPI برای خوانایی بهتر مسیرها و امکان اتصال ابزارهای دیگر [۱۰].

این هدف‌ها فقط به ظاهر سامانه مربوط نیستند. کاتالوگ و مستندات بخش عمومی محصول را می‌سازند؛ ورود، اشتراک و مصرف بخش خصوصی را شکل می‌دهند؛ و OpenAPI کمک می‌کند قرارداد سرویس هم برای انسان خوانا باشد و هم برای ابزارهای بیرونی قابل استفاده بماند.

4-1 دامنه و محدودیت‌ها

دامنه گزارش همان نسخه فعلی پروژه است: وب‌اپلیکیشن فارسی، API بک‌اند، اجرای محلی و جریان‌های اصلی کاربر. اجرای محلی با Docker Compose هم در همین دامنه قرار می‌گیرد، چون برای بالا آوردن چند سرویس کنار هم طراحی شده است [۸].

در کنار قابلیت‌ها، چند محدودیت هم باید صریح گفته شود. تنظیمات تولیدی هنوز کامل نیست، بخشی از رفتارها برای محیط توسعه مناسب‌ترند، پرداخت واقعی و اتصال کامل به سرویس‌های بیرونی نیاز به سخت‌سازی دارد و آزمون مرورگر باید منظم‌تر در چرخه ارزیابی قرار بگیرد. بنابراین گزارش، پروژه را به عنوان یک نمونه کامل و قابل توسعه بررسی می‌کند، نه محصولی که همین حالا برای بهره‌برداری عمومی آماده باشد.

5-1 ذی‌نفعان

ذی‌نفعان اصلی پروژه شامل کاربر مهمان، توسعه‌دهنده احرازشده، مدیر سیستم، ارائه‌دهنده ورود اجتماعی، سرویس مقصد API و ابزارهای بیرونی مانند RapidAPI هستند [۱۱]. انتظار این گروه‌ها یکسان نیست: مهمان به شناخت سریع سرویس نیاز دارد، توسعه‌دهنده به ابزار و داده خصوصی، مدیر به کنترل سامانه، و سرویس‌های بیرونی به قراردادی قابل اعتماد.

6-1 سازمان‌دهی گزارش

ادامه گزارش به همین ترتیب پیش می‌رود: ابتدا فناوری‌های استفاده‌شده معرفی می‌شوند، سپس نیازمندی‌ها و معماری بررسی می‌شوند. بعد از آن، پیاده‌سازی، آزمون، نتایج و در پایان مسیر ادامه

کار توضیح داده می‌شود.

7-1 جمع‌بندی فصل

در این فصل روشن شد که پروژه فقط یک تمرین بک‌اند یا یک رابط کاربری ساده نیست؛ یک پلتفرم اشتراک‌گذاری و مصرف API است. به همین دلیل، گزارش باید هم تجربه کاربر را جدی بگیرد و هم قرارداد فنی میان فرانت‌اند، بک‌اند و داده را دقیق توضیح دهد.

فصل ۲

فناوری‌ها و پیشینه

انتخاب فناوری در این پروژه فقط بر اساس رایج بودن ابزارها انجام نشده است. هر بخش باید با نقش خود در محصول هماهنگ باشد: بک‌اند باید قرارداد API را روشن نگه دارد، فرانت‌اند باید تجربه فارسی و سریع بسازد، و اجرای محلی باید برای ارائه و آزمون در دسر اضافه ایجاد نکند. بنابراین معرفی فناوری‌ها در ادامه، بیشتر روی نقش آن‌ها در خود محصول تمرکز دارد.

1-2 Django، Python و Django REST Framework

Python زبان اصلی بک‌اند است؛ زبانی خوانا و پرکاربرد که برای توسعه وب و ساخت سرویس‌های داده‌محور انتخاب مناسبی به شمار می‌آید [۱]. روی این زبان، Django چارچوب اصلی وب را فراهم می‌کند و بخش‌هایی مثل تنظیمات، مسیرها، امنیت پایه، مدیریت کاربران، پنل مدیریت و ابزارهای آزمون را در اختیار پروژه می‌گذارد [۲].

Django REST Framework نقش تکمیل‌کننده همین ساختار را دارد. مفاهیمی مثل Serializer، View، اعتبارسنجی ورودی و پاسخ JSON کمک می‌کنند مسیرهای عمومی و خصوصی سامانه با قرارداد منظم‌تری نوشته شوند [۳]. در این پروژه، همین ترکیب پایه ساخت کاتالوگ، حساب کاربری، اشتراک، مصرف و ابزارهای توسعه‌دهنده است.

2-2 MongoDB و مدل داده

داده‌های این پروژه شکل کاملاً یکسانی ندارند. اطلاعات کاربر، نشست، API، پلن، مصرف و جریان Studio هر کدام جزئیات خودشان را دارند. به همین دلیل، مدل سندی برای این فضا طبیعی‌تر است. MongoDB نیز در داکيومنت رسمی خود همین رویکرد سندمحور را برای ذخیره داده‌های انعطاف‌پذیر توضیح می‌دهد [۹]. در گزارش، وقتی از لایه داده صحبت می‌شود، منظور بخشی است که همین داده‌های محصول را برای مسیرهای بک‌اند آماده می‌کند.

این انتخاب برای بازارچه API منطقی است. مستندات یک سرویس، پلن‌های قیمت‌گذاری، رخدادهای مصرف و جریان‌های Studio همیشه مثل رکوردهای ساده و ثابت رفتار نمی‌کنند. مدل سندی باعث می‌شود این تفاوت‌ها با انعطاف بیشتری نگهداری شوند.

3-2 React، TypeScript و Vite

فرانت‌اند با React ساخته شده است؛ کتابخانه‌ای که برای ساخت رابط‌های کاربری مبتنی بر کامپوننت استفاده می‌شود [۴]. برای پروژه‌ای که صفحه‌های متنوعی مثل کاتالوگ، جزئیات سرویس، Dashboard و Studio دارد، این مدل کمک می‌کند اجزای تکرارشونده جدا و قابل نگهداری بمانند. TypeScript هم به کد سمت کاربر نوع‌دهی اضافه می‌کند تا خطاهای ساده زودتر دیده شوند و قرارداد بین بخش‌های مختلف روشن‌تر بماند [۵].

Vite ابزار توسعه و build فرانت‌اند است. داکيومنت رسمی Vite روی سرعت راه‌اندازی و تجربه توسعه سریع تاکید دارد [۶]. این ویژگی در کار روزمره پروژه مهم است، چون تغییر در صفحه‌ها و کامپوننت‌ها باید سریع دیده شود و توسعه رابط کاربری کند نشود.

4-2 Node.js و ابزارهای فرانت‌اند

اکوسیستم فرانت‌اند پروژه روی Node.js و ابزارهای بسته‌بندی جاوااسکریپت می‌چرخد. Node.js اجرای جاوااسکریپت خارج از مرورگر را فراهم می‌کند و پایه بسیاری از ابزارهای توسعه فرانت‌اند است [۷]. کتابخانه‌هایی مثل TanStack Query، Axios و مجموعه‌های کامپوننتی، کار ارتباط با بک‌اند، مدیریت وضعیت درخواست‌ها و ساخت فرم‌ها و جدول‌ها را ساده‌تر می‌کنند.

در گزارش، نام این ابزارها بدون ترجمه آمده است؛ چون برای بسیاری از آن‌ها معادل فارسی دقیق و جاافتاده‌ای وجود ندارد و ترجمه اجباری، متن را کم‌دقت‌تر می‌کند.

5-2 Docker Compose و اجرای محلی

Docker Compose برای تعریف و اجرای چند سرویس کنار هم استفاده می‌شود [۸]. در این پروژه، همین ابزار باعث می‌شود بک‌اند و فرانت‌اند کنار هم بالا بیایند و مسیر ارتباطی مشخصی داشته باشند. این روش برای ارائه، آزمون دستی و توسعه محلی مناسب است، هرچند جای پیکربندی تولیدی را نمی‌گیرد.

6-2 OpenAPI و RapidAPI

OpenAPI برای توصیف قرارداد API به کار می‌رود؛ یعنی مسیرها، ورودی‌ها، خروجی‌ها و پاسخ‌ها را به شکلی می‌نویسد که هم انسان و هم ابزار بتوانند آن را بخوانند [۱۰]. RapidAPI نیز نمونه شناخته‌شده‌ای از فضای بازارچه API است و برای فهم تجربه کاربر در کشف، مقایسه و مصرف API مرجع خوبی به شمار می‌آید [۱۱].

7-2 جمع‌بندی فصل

پشته فنی پروژه با ماهیت آن هماهنگ است: Django REST Framework برای ساخت API، React برای تجربه کاربری، MongoDB برای داده‌های سندی، Docker Compose برای اجرای محلی، و OpenAPI برای خوانایی قرارداد سرویس. نتیجه این ترکیب، گزارشی است که فقط درباره ابزارها حرف نمی‌زند؛ بلکه نشان می‌دهد هر ابزار چه نقشی در شکل‌گیری محصول دارد.

فصل ۳

تحلیل نیازمندی‌ها

نیازمندی‌های این پروژه از دل تجربه کاربر بیرون می‌آیند. کاربر مهمان می‌خواهد سرویس‌ها را راحت پیدا کند؛ توسعه‌دهنده می‌خواهد بعد از ورود، حساب، مصرف و ابزارهای خود را مدیریت کند؛ مدیر هم باید بتواند سامانه را کنترل و نگهداری کند. بنابراین نیازمندی‌ها فقط یک فهرست اداری نیستند، بلکه توضیح می‌دهند هر بخش از سامانه باید چه مشکلی را حل کند.

1-3 نقش‌های کاربری

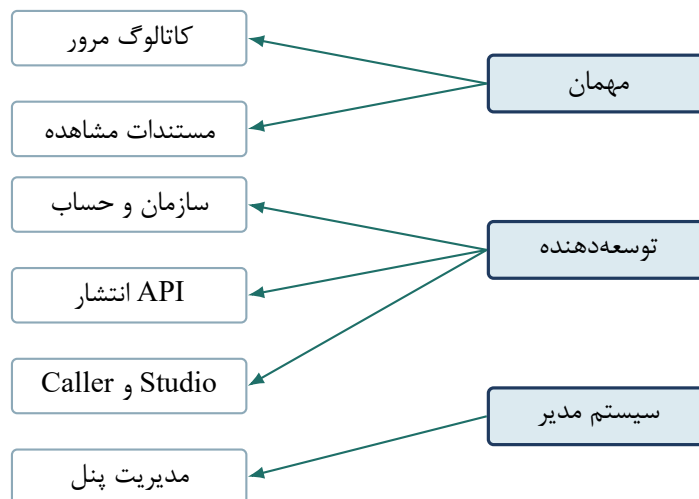
سه نقش اصلی در پروژه دیده می‌شود: مهمان، توسعه‌دهنده احراز شده و مدیر سیستم. مهمان با بخش عمومی کاتالوگ سروکار دارد. توسعه‌دهنده بعد از ورود، وارد فضای کاری خود می‌شود و با Dashboard، انتشار API، Caller و Studio کار می‌کند. مدیر سیستم هم مسئول کنترل کلی و نگهداری داده‌هاست.

مهمان بدون ورود می‌تواند بخش عمومی بازارچه را ببیند؛ این بخش برای معرفی API‌ها و ایجاد اعتماد اولیه ضروری است. توسعه‌دهنده پس از احراز هویت به امکاناتی می‌رسد که با داده خصوصی او ارتباط دارند، مانند کلید API، سازمان، اشتراک، مصرف، انتشار و ابزارهای آزمایشی. نقش مدیر سیستم بیشتر به نگهداری داده و پشتیبانی از مسیرهای مدیریتی مربوط می‌شود و به همین دلیل از نظر سطح دسترسی حساس‌تر است.

2-3 نیازمندی‌های عملکردی و غیرفعال

جدول ۳-۱ نیازمندی‌ها را در دو گروه اصلی نشان می‌دهد. نیازمندی‌های عملکردی همان کارهایی هستند که کاربر مستقیم با آن‌ها روبه‌رو می‌شود؛ مثل جستجوی API، ورود، انتشار یا مشاهده مصرف. نیازمندی‌های امنیتی و اجرایی کیفیت همین رفتارها را حفظ می‌کنند؛ مثل پنهان کردن کلیدها، پاسخ خطای قابل فهم و اجرای محلی با Docker Compose.

جدول ۳-۱ نیازمندی‌های اصلی پروژه



شکل ۱-۳ نمای نقش‌ها و جریان‌های اصلی پروژه

شناسه	نیازمندی	نوع	اولویت	وضعیت
R1	نمایش فهرست API ها با صفحه بندی، جستجو، دسته بندی و مرتب سازی	عملکردی	زیاد	انجام شده
R2	نمایش جزئیات، پلن ها، مستندات، endpoint ها و API های مشابه	عملکردی	زیاد	انجام شده
R3	ثبت نام، ورود، نشست، خروج و کشف ارائه دهندگان ورود اجتماعی	عملکردی	زیاد	انجام شده
R4	مدیریت کاربر، پروفایل، سازمان، اشتراک، پرداخت دستی و مصرف	عملکردی	زیاد	انجام شده
R5	انتشار API توسط کاربر احراز شده	عملکردی	متوسط	انجام شده
R6	اجرای درخواست نمونه در Caller و ثبت مصرف	عملکردی	متوسط	انجام شده
R7	استقرار جریان Studio و ثبت مصرف	عملکردی	متوسط	انجام شده
R8	بازگشت خطای یکنواخت برای اعتبارسنجی و دسترسی	غیرفعال	زیاد	انجام شده
R9	پنهان سازی کلیدها و توکن ها در پاسخ ها و لاگ ها	امنیتی	زیاد	انجام شده

شناسه	نیازمندی	نوع	اولویت	وضعیت
R10	اجرای محلی دو سرویس با استقرار Docker Compose	متوسط	مستند	

اولویت زیاد برای نیازهایی در نظر گرفته شده که نبود آنها هسته محصول را مختل می‌کند؛ مانند کاتالوگ، احراز هویت، حساب کاربری و امنیت داده حساس. اولویت متوسط برای قابلیت‌هایی است که ارزش محصول را کامل‌تر می‌کنند، اما می‌توانند در نسخه‌های بعدی هم رشد کنند؛ مانند Studio، Caller و اجرای محلی.

3-3 ورودی‌ها، خروجی‌ها و اعتبارسنجی

ورودی‌های اصلی شامل مشخصات حساب، پروفایل، شناسه یا نامک API، فیلترهای جستجو، امتیاز، داده انتشار API، شناسه پلن اشتراک و گره‌های Studio است. در معماری مبتنی بر Django REST Framework، معمولاً Serializer همین داده‌ها را کنترل، پاک‌سازی و به شکل قابل استفاده تبدیل می‌کند [۳].

خروجی‌های اصلی سامانه در قالب JSON بازگردانده می‌شوند و برای مصرف فرانت‌اند طراحی شده‌اند. پاسخ‌های عمومی معمولاً داده کاتالوگ، مستندات، پلن‌ها و وضعیت سلامت را شامل می‌شوند. پاسخ‌های خصوصی، اطلاعاتی مانند پروفایل، سازمان‌ها، کلید پنهان‌شده، اشتراک و تاریخچه مصرف را برمی‌گردانند. جداسازی این دو دسته مهم است، چون داده عمومی برای همه کاربران قابل نمایش است، اما داده خصوصی باید فقط بعد از احراز هویت برگردد.

اعتبارسنجی در این پروژه دو کار اصلی انجام می‌دهد: جلوی ذخیره داده ناقص یا نامعتبر را می‌گیرد و خطا را به پاسخی قابل فهم برای Client تبدیل می‌کند. برای نمونه، هنگام ثبت امتیاز باید مقدار امتیاز معتبر باشد، هنگام انتشار API فیلدهای اصلی باید حاضر باشند و هنگام اجرای Studio ساختار گره‌ها باید قابل پردازش باشد.

4-3 امنیت

احراز هویت باید طوری انجام شود که کاربر بعد از ورود به بخش‌های خصوصی برسد، اما داده حساس او در پاسخ‌ها آشکار نشود. در پروژه‌ای که با API Key، توکن و کوکی کار می‌کند، این موضوع فقط یک جزئیات جانبی نیست؛ بخشی از امنیت اصلی محصول است [۲، ۳].

از دید نیازمندی، امنیت فقط ورود کاربر نیست. کنترل دسترسی، نگهداری نشست، جلوگیری از افشای رازها و محدود کردن خروجی‌های حساس هم بخشی از همین موضوع‌اند. آزمون‌های موجود نشان می‌دهند که پنهان‌سازی داده حساس در خروجی‌ها بررسی شده است. با این حال، برای محیط تولید باید رازهای پیکربندی از فایل تنظیمات خارج شوند و سیاست‌های سخت‌گیرانه‌تری برای کوکی، دامنه مجاز و خطاهای عملیاتی اعمال شود.

5-3 جمع‌بندی فصل

نیازهای پروژه حول کشف عمومی API، فضای کاری توسعه‌دهنده، مدیریت حساب و اشتراک، و امنیت نشست شکل گرفته‌اند. همین نیازها مبنای فصل معماری هستند؛ چون لایه‌های سامانه باید بتوانند نقش‌ها، جریان‌ها و محدودیت‌های امنیتی را پشتیبانی کنند.

فصل ۴

معماری و طراحی

معماری پروژه را می‌توان از مسیر یک درخواست ساده فهمید. کاربر از رابط React شروع می‌کند، درخواست به بک‌اند Django REST Framework می‌رسد، داده اعتبارسنجی می‌شود و سپس لایه داده نتیجه را آماده می‌کند. توضیح معماری در اینجا بر همین مسیر تکیه دارد، بدون این‌که خواننده را درگیر نام‌های داخلی فایل‌ها کند.

1-4 نمای کلی

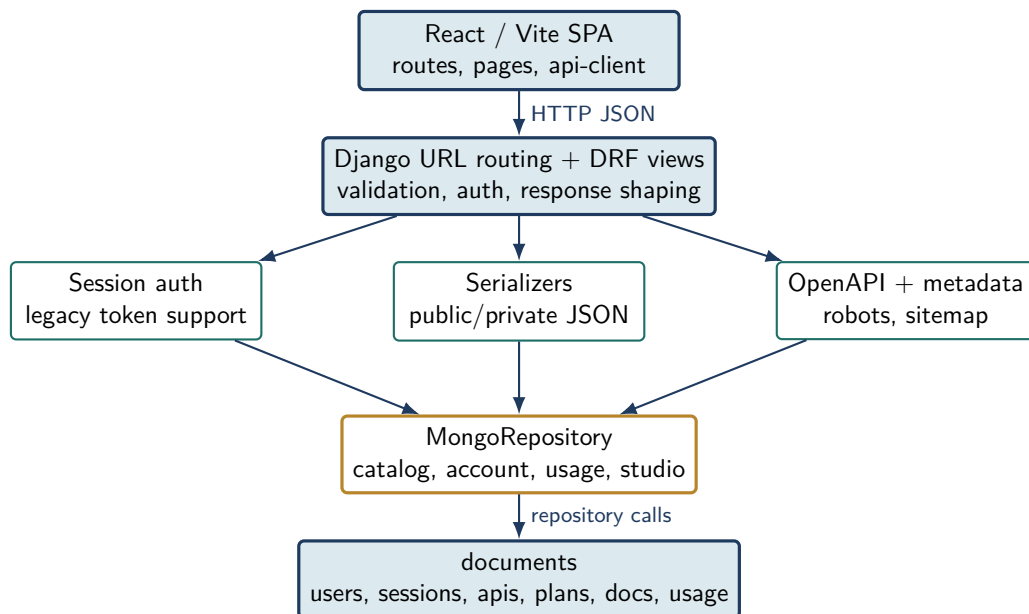
معماری پروژه سه لایه اصلی دارد: رابط کاربری React، بک‌اند Django REST Framework و لایه داده. این مدل با الگوی رایج وب‌اپلیکیشن‌های مدرن هماهنگ است؛ فرانت‌اند چیزی را می‌سازد که کاربر می‌بیند، بک‌اند قرارداد API را مدیریت می‌کند و لایه داده مسئول نگهداری اطلاعات محصول است [۴، ۳، ۹].

در این طراحی، کاربر نهایی بیشتر با یک برنامه تک‌صفحه‌ای روبه‌رو است، اما عملیات داده از مسیر API نسخه‌دار انجام می‌شود. نسخه‌دار بودن مسیرها برای نگهداری آینده اهمیت دارد، چون اجازه می‌دهد قراردادهای جدید اضافه شوند بدون این‌که Client‌های قدیمی ناگهان از کار بیفتند. نمایش پوسته SPA برای مسیرهای ناشناخته فرانت‌اند نیز با مسیریابی سمت Client هماهنگ است.

2-4 نمودار معماری

در شکل ۴-۱، مسیر اصلی ارتباط دیده می‌شود: فرانت‌اند درخواست‌ها را به API بک‌اند می‌فرستد، بک‌اند ورودی را اعتبارسنجی می‌کند، پاسخ را به قالب JSON درمی‌آورد و کار با داده را به لایه Repository می‌سپارد.

در این معماری، هر لایه مسئولیت مشخصی دارد. فرانت‌اند نمایش، فرم‌ها و حرکت کاربر بین صفحه‌ها را مدیریت می‌کند. بک‌اند احراز هویت، اعتبارسنجی و شکل پاسخ را بر عهده دارد. Repository هم جزئیات ذخیره‌سازی و بازیابی داده را پشت یک مرز روشن نگه می‌دارد. این جداسازی کمک می‌کند تغییر در ظاهر کاربر، منطق داده را بی‌دلیل درگیر نکند.



شکل ۴-۱ معماری سطح بالای پروژه

3-4 مدل داده

مدل داده پروژه حول چند گروه اصلی می‌چرخد: هویت کاربر، کاتالوگ API، دسترسی و مصرف، فضای کاری توسعه‌دهنده و اشتراک. این تقسیم‌بندی با مدل سندی MongoDB سازگار است، چون هر گروه می‌تواند ساختار و جزئیات خودش را داشته باشد [۹].

جدول ۴-۱ گروه‌های داده اصلی

گروه	مجموعه‌ها و مسئولیت
هویت	users, sessions و legacy_tokens برای کاربر، نشست و توکن سازگار
کاتالوگ	categories, apis, pricing_plans, documentations, api_endpoints و api_ratings
مصرف	access_grants و api_usage برای مجوز دسترسی و رخدادهای مصرف

گروه	مجموعه‌ها و مسئولیت
فضای کاری	organizations و studio_flows برای امکانات توسعه‌دهنده
اشتراک	subscription_plans، user_subscriptions و subscription_checkouts

گروه هویت پایه دسترسی کاربر را می‌سازد و به نشست‌ها و توکن‌های سازگار قدیمی متصل است. گروه کاتالوگ محتوای عمومی بازارچه را نگه می‌دارد؛ یعنی دسته‌ها، خود API‌ها، پلن قیمت‌گذاری، مستندات، نقاط انتهایی و امتیازها. گروه مصرف و اشتراک داده‌هایی را ثبت می‌کند که برای داشبورد، کنترل دسترسی و گزارش مصرف لازم‌اند. گروه فضای کاری نیز قابلیت‌های مخصوص توسعه‌دهنده مانند سازمان و جریان Studio را پشتیبانی می‌کند.

4-4 مرزهای خارجی

مرزهای خارجی پروژه شامل ارائه‌دهندگان ورود اجتماعی، سرویس مقصد API و بازارهای بیرونی مانند RapidAPI است [۱۱]. این مرزها باید روشن باشند، چون هر کدام سطحی از ریسک و وابستگی بیرونی وارد سامانه می‌کنند.

شناخت مرز خارجی برای تحلیل ریسک مهم است. ورود اجتماعی به پیکربندی ارائه‌دهنده وابسته است و در صورت نبود auth_url نمی‌تواند کامل اجرا شود. Caller و Studio از نظر محصول به ارتباط با سرویس‌های بیرونی نزدیک‌اند، اما در وضعیت فعلی بیشتر برای نمایش جریان و ثبت مصرف استفاده می‌شوند. بنابراین گزارش میان قابلیت پیاده‌سازی شده و اتصال تولیدی واقعی تفاوت می‌گذارد.

5-4 جمع‌بندی فصل

معماری پروژه یک SPA متصل به API بک‌اند است. این طراحی برای نسخه فعلی مناسب است و فضای پروژه را به یک پلتفرم واقعی API نزدیک می‌کند. برای استفاده تولیدی، تصمیم‌هایی درباره پایگاه داده، مدیریت رازها و اتصال‌های بیرونی باید دقیق‌تر شود.

فصل ۵

پیاده‌سازی

در پیاده‌سازی، مهم‌ترین نکته این است که کاربر از بیرون چه رفتاری می‌بیند و پشت آن رفتار، چه بخش‌هایی با هم کار می‌کنند. به همین دلیل، متن به جای تکیه بر نام فایل‌ها و مسیرهای داخلی، مسیرهای اصلی محصول را توضیح می‌دهد: از کاتالوگ و ورود گرفته تا Dashboard، انتشار API، Caller و Studio.

1-5 مسیرهای بک‌اند

بک‌اند مجموعه‌ای از مسیرهای نسخه‌دار دارد که بخش‌های اصلی محصول را پوشش می‌دهند: سلامت سامانه، قرارداد OpenAPI، ثبت‌نام، ورود، خروج، حساب کاربری، پروفایل، API Key، سازمان‌ها، اشتراک، مصرف، کاتالوگ، مستندات، Endpointها، Caller و Studio. نسخه‌دار بودن مسیرها کمک می‌کند توسعه آینده کنترل‌شده‌تر باشد و Clientها با تغییرات ناگهانی روبه‌رو نشوند [۳، ۱۰].

این مسیرها دو سطح تجربه می‌سازند. سطح اول عمومی است و برای مشاهده کاتالوگ، جزئیات API و مستندات استفاده می‌شود. سطح دوم خصوصی است و بعد از ورود کاربر فعال می‌شود؛ مثل Dashboard، مصرف، اشتراک، سازمان و انتشار API. همین جداسازی باعث می‌شود کاربر مهمان با داده خصوصی روبه‌رو نشود و توسعه‌دهنده بعد از ورود، فضای کاری خودش را ببیند.

2-5 لایه Repository

لایه Repository منطق کار با داده را متمرکز می‌کند. ساخت کاربر، نگهداری نشست، چرخش API Key، خواندن کاتالوگ، ثبت مصرف Caller و ذخیره جریان‌های Studio از همین مسیر انجام می‌شود. این شیوه باعث می‌شود Viewها بیش از حد شلوغ نشوند و جزئیات داده در یک لایه مشخص بماند.

در پروژه‌های شبیه بازارچه API، این جداسازی اهمیت زیادی دارد. داده‌های حساب، کاتالوگ، مستندات، اشتراک و مصرف شکل یکسانی ندارند. وقتی این بخش‌ها پشت یک Repository قرار می‌گیرند، تغییر در روش ذخیره‌سازی کمتر به بخش‌های دیگر منت می‌کند.

3-5 Serializer و قرارداد پاسخ

Serializer ها مرز امن بین داده داخلی و خروجی قابل نمایش هستند. داده‌ای که داخل سامانه نگهداری می‌شود، همیشه نباید با همان شکل به کاربر برگردد. برای نمونه، مقدار کامل API Key نباید در پاسخ عادی دیده شود و بهتر است فقط نسخه پنهان شده آن نمایش داده شود. این رویکرد با نقش Serializer در Django REST Framework هماهنگ است [۳].

در بخش انتشار API نیز Serializer کمک می‌کند داده ورودی قبل از ثبت بررسی شود: نام، آدرس پایه، روش احراز هویت، دسته و توضیح سرویس باید روشن باشند. نتیجه این کار، پاسخ‌هایی است که برای فرانت‌اند قابل پیش‌بینی‌تر و برای کاربر قابل فهم‌ترند.

4-5 فرانت‌اند

فرانت‌اند همان جایی است که کاربر بیشترین تماس را با محصول دارد: صفحه مرور API ها، جزئیات سرویس، مستندات، قیمت‌گذاری، پرداخت، ورود، ثبت‌نام، Dashboard، سازمان‌ها، انتشار، Caller و Studio. معماری کامپوننتی React برای چنین تجربه‌ای مناسب است، چون بخش‌های تکرارشونده مثل فرم، کارت، جدول و وضعیت بارگذاری را می‌توان جدا نگه داشت [۴].

ارتباط فرانت‌اند با بک‌اند از طریق Client های مشخص انجام می‌شود. یک گروه درخواست‌ها برای کاتالوگ است، یک گروه برای احراز هویت، و یک گروه برای حساب کاربری و ابزارهای توسعه‌دهنده. این جداسازی باعث می‌شود صفحه‌ها کمتر به جزئیات آدرس‌ها وابسته باشند و نگهداری پروژه ساده‌تر شود.

5-5 جریان‌های اصلی پیاده‌سازی

جدول ۵-۱ نشان می‌دهد هر جریان مهم از نگاه کاربر چه معنایی دارد و در پشت صحنه کدام بخش‌ها را درگیر می‌کند.

جدول ۵-۱ جریان‌های اصلی در پیاده‌سازی

جریان	بخش‌های درگیر	خروجی برای کاربر
مرور کاتالوگ	فرانت‌اند، Client کاتالوگ، بک‌اند و لایه داده	فهرست قابل جستجو و صفحه‌بندی شده API ها
جزئیات API	صفحه جزئیات، مستندات، پلن‌ها و Endpoint ها	شناخت سرویس قبل از مصرف
ورود و نشست	فرم ورود، احراز هویت، کوکی نشست و پاسخ کاربر	دسترسی به بخش خصوصی

جریان	بخش‌های درگیر	خروجی برای کاربر
Dashboard	حساب کاربری، مصرف، اشتراک، سازمان و API Key	مدیریت وضعیت توسعه‌دهنده
انتشار API	فرم انتشار، اعتبارسنجی و ثبت در کاتالوگ	اضافه‌شدن سرویس جدید به محصول
Studio و Caller	ابزار تست، جریان کاری و ثبت مصرف	آزمایش و طراحی جریان‌های مرتبط با API

6-5 بخش‌های فنی

جدول ۳-۵ نقش بخش‌های فنی را با زبان قابل خواندن خلاصه می‌کند، بدون این‌که گزارش به فهرست نام‌های داخلی پروژه تبدیل شود.

جدول ۳-۵ بخش‌های فنی و نقش آن‌ها در پروژه

بخش	نقش در سامانه	اثر در گزارش
مسیریابی بک‌اند	تعریف سطح API و نسخه‌بندی مسیرها	توضیح قرارداد ارتباطی سامانه
Serializer و View	دریافت درخواست، اعتبارسنجی و پاسخ	توضیح رفتار حساب، کاتالوگ و ابزارها
Repository داده	مدیریت داده‌های کاربر، کاتالوگ، مصرف و اشتراک	توضیح مدل عملیاتی داده
رابط کاربری	ساخت صفحه‌ها و حرکت کاربر	توضیح تجربه محصول
Client فرانت‌اند	ارتباط کنترل‌شده با بک‌اند	توضیح اتصال صفحه‌ها به API
اجرای محلی	بالا آوردن فرانت‌اند و بک‌اند کنار هم	توضیح مسیر ارائه و آزمون دستی
آزمون‌ها	کنترل رفتارهای اصلی و خطاهای مهم	پشتوانه بخش ارزیابی

7-5 نمونه کد

نمونه زیر بخش کوچکی از منطق پنهان کردن داده حساس را نشان می‌دهد.

```
1 SENSITIVE_KEYS = {"password", "api_key", "token", "secret", "
    authorization"}
2
3 def redact_secrets(value):
4     if isinstance(value, dict):
5         return {
6             key: "[REDACTED]" if str(key).lower() in
              SENSITIVE_KEYS else redact_secrets(item)
7         for key, item in value.items()
8     }
9     return value
```

این قطعه کوتاه است، اما برای چنین پروژه‌ای اهمیت زیادی دارد. سامانه با گذرواژه، API Key، توکن و اطلاعات احراز هویت سروکار دارد. اگر این داده‌ها بدون کنترل وارد پاسخ‌ها یا لاگ‌ها شوند، امنیت کاربر آسیب می‌بیند. جایگزین کردن مقدار حساس با [REDACTED] کمک می‌کند خروجی‌ها برای بررسی فنی قابل استفاده باشند، اما رازهای کاربر آشکار نشوند.

8-5 جمع‌بندی فصل

پیاده‌سازی پروژه از چند بخش روشن ساخته شده است: فرانت‌اند برای تجربه کاربر، بک‌اند برای قرارداد API، Serializer برای کنترل ورودی و خروجی، و Repository برای کار با داده. کنار هم قرار گرفتن این بخش‌ها باعث می‌شود پروژه از یک نمونه ساده فراتر برود و به یک پلتفرم واقعی اشتراک‌گذاری API نزدیک شود.

فصل ۶

آزمون و ارزیابی

بعد از پیاده‌سازی، مهم‌ترین پرسش این است که آیا مسیرهای اصلی سامانه واقعاً پایدار کار می‌کنند یا نه. در این بخش، نتیجه آزمون‌های بک‌اند آمده و مشخص شده بخش‌هایی مثل احراز هویت، کاتالوگ، مصرف، امنیت خروجی و قرارداد عمومی API چگونه بررسی شده‌اند.

1-6 راهبرد آزمون

آزمون‌های بک‌اند چند بخش را پوشش می‌دهند: آماده‌بودن برنامه، پنهان‌سازی رازها، اعتبارسنجی داده، احراز هویت، کاتالوگ، مستندات، امتیازدهی، انتشار API، Caller، Studio، سازمان‌ها، اشتراک، پروفایل، طرح OpenAPI و مسیرهای متادیتای سایت. چارچوب آزمون در Django برای همین نوع کنترل رفتار سمت سرور استفاده می‌شود [۲].

راهبرد آزمون ترکیبی است. بخشی از آزمون‌ها رفتار زیرساختی را بررسی می‌کنند؛ مانند آماده‌بودن برنامه، شاخص‌ها و ترتیب برنامه‌های نصب‌شده. بخشی دیگر به رفتار محصول مربوط است؛ مانند ثبت‌نام، ورود، مشاهده کاتالوگ، انتشار API و مدیریت اشتراک. گروه سوم آزمون‌ها روی امنیت و قرارداد خروجی تمرکز دارد؛ مانند پنهان‌سازی رازها، محدودبودن مسیرهای خصوصی و معتبر بودن طرح OpenAPI.

2-6 نتایج اجرا

دستور زیر در ریشه پروژه اجرا شد:

```
python manage.py test api
```

نتیجه اجرا این بود که 49 آزمون اجرا شد و همه با وضعیت موفق پایان یافتند. پایگاه آزمون SQLite نیز به‌صورت خودکار ساخته و بعد از پایان کار حذف شد.

این نتیجه نشان می‌دهد بک‌اند در محدوده آزمون‌های موجود پایدار است. البته باید روشن گفت که این موفقیت، همه جنبه‌های محصول را اثبات نمی‌کند. برای ادعای آمادگی تولید، آزمون‌های

بیشتری مانند بار، امنیت، مرورگر و اجرای کامل Docker Compose باید به چرخه ارزیابی اضافه شود.

3-6 جدول آزمون‌ها

جدول ۱-۶ خلاصه آزمون‌های اجراشده

شناسه	مؤلفه	سناریو	انتظار	نتیجه واقعی	وضعیت
T1	امنیت	حذف گذرواژه، کلید و توکن از داده	رازها دیده نشوند	موفق	قبول
T2	مدل‌ها	نامک فارسی یکتا و اعتبار رنگ و امتیاز	خطا برای داده نامعتبر	موفق	قبول
T3	احراز هویت	ثبت نام، ورود، نشست و خروج	نشست ساخته و پاک شود	موفق	قبول
T4	کاتالوگ	فهرست، جزئیات، مستندات و endpoint	داده صفحه‌بندی	موفق	قبول
T5	حساب	پروفایل، کلید API، سازمان و دسترسی	مسیر خصوصی با احراز هویت	موفق	قبول
T6	اشتراک	ساخت، مشاهده، لغو و تأیید پرداخت دستی	وضعیت درست checkout	موفق	قبول
T7	Studio Caller	ثبت مصرف پس از اجرا یا استقرار	رکورد مصرف ساخته شود	موفق	قبول
T8	طرح و سایت	OpenAPI، robots.txt و sitemap.xml	پاسخ معتبر	موفق	قبول

جدول نشان می‌دهد پوشش آزمون‌ها به یک قابلیت محدود نیست و بیشتر مسیرهای اصلی محصول را لمس می‌کند. وجود آزمون برای مسیرهای OpenAPI و متادیتای سایت نیز اهمیت

دارد، زیرا این مسیرها مستقیماً به مصرف‌کنندگان بیرونی و موتورهای جستجو مربوط می‌شوند. آزمون‌های امنیتی هم بررسی می‌کنند داده‌های حساس در پاسخ عادی سامانه آشکار نشوند.

4-6 محدودیت‌های آزمون

آزمون فرانت‌اند با Playwright در این اجرا دوباره انجام نشد. بنابراین نتیجه قطعی این فصل مربوط به بک‌اند است. برای کامل‌شدن ارزیابی، جریان‌هایی مثل ورود، جستجو، مشاهده جزئیات API، Dashboard و Studio باید در مرورگر هم اجرا شوند [۱۲].

این محدودیت یعنی رفتار تصویری و تعاملی فرانت‌اند در این گزارش دوباره اثبات نشده است. برای مثال، درست‌بودن نمایش فرم‌ها، حرکت بین صفحات، پیام‌های خطا، واکنش‌گرایی و اتصال واقعی صفحه‌ها به بک‌اند باید با اجرای تازه آزمون مرورگر یا بررسی دستی ثبت شود. بنابراین ارزیابی فعلی از نظر بک‌اند قوی‌تر از فرانت‌اند است.

5-6 جمع‌بندی فصل

آزمون بک‌اند اجرا شد و همه 49 آزمون موفق بودند. برای تکمیل ارزیابی، اجرای تازه آزمون‌های فرانت‌اند، سنجش مسیر Docker Compose، افزودن آزمون بار سبک و بررسی امنیت پیکربندی پیشنهاد می‌شود.

فصل ۷

نتایج و بحث

در این بخش، پروژه از زاویه نتیجه نهایی بررسی می‌شود: چه چیزی آماده شده، کدام بخش‌ها برای ارائه قابل اتکا هستند و چه کارهایی هنوز باید کامل شوند. هدف، ارائه تصویر واقعی‌تری از وضعیت سامانه است، نه فقط آوردن فهرستی از قابلیت‌ها.

1-7 ویژگی‌های پیاده‌سازی شده

ویژگی‌های اصلی پروژه شامل کاتالوگ عمومی، صفحه جزئیات API، مستندات، پلن‌ها، ثبت امتیاز، ثبت‌نام، ورود، نشست، خروج، پروفایل، چرخش API Key، سازمان‌ها، اشتراک، checkout دستی، مصرف، Caller، Studio و انتشار API است. کنار هم قرار گرفتن این بخش‌ها، پروژه را به فضای API Marketplace نزدیک می‌کند [۱۱].

این ویژگی‌ها دو نوع ارزش ایجاد می‌کنند. ارزش نخست برای مصرف‌کننده API است: کشف، مقایسه، مشاهده مستندات و انتخاب پلن. ارزش دوم برای توسعه‌دهنده یا مالک API است: مدیریت حساب، سازمان، کلید، انتشار، مشاهده مصرف و آزمایش جریان‌ها. همین دوغانگی باعث می‌شود پروژه از یک فهرست ساده فراتر برود و به شکل یک بازارچه عملیاتی نزدیک شود.

2-7 نمونه خروجی API

طرح OpenAPI برای معرفی مسیرها و پاسخ‌ها استفاده می‌شود. نمونه زیر پاسخ ساده مسیر سلامت را نشان می‌دهد [۱۰]:

```
1 GET /api/v1/system/health/
2 {
3   "status": "ok",
4   "timestamp": "2026-05-17T00:00:00Z"
5 }
```

این پاسخ کوتاه است، اما برای ارزیابی اهمیت دارد؛ زیرا نشان می‌دهد مسیر نسخه‌دار API فعال است و Client می‌تواند وضعیت سرویس را بدون ورود بررسی کند. وجود زمان در پاسخ نیز

برای عیب‌یابی و مانیتورینگ اولیه مفید است.

3-7 نقاط قوت

نقاط قوت پروژه را می‌توان در چند مورد خلاصه کرد: مسیرهای نسخه‌دار، پاسخ خطای یکنواخت، آزمون‌های متعدد، پنهان کردن رازها، پشتیبانی از فارسی، قرارداد OpenAPI و اجرای محلی با Docker Compose.

از نظر نگهداری، مهم‌ترین نقطه قوت پروژه جداسازی لایه‌هاست. مسیرها، Serializerها، لایه داده و Client فرانت‌اند نقش‌های نسبتاً روشن دارند. از نظر تجربه کاربر، پشتیبانی از راست‌به‌چپ و فارسی باعث می‌شود سامانه برای مخاطب ایرانی مناسب‌تر باشد. از نظر توسعه، وجود آزمون‌ها و طرح OpenAPI مسیر ادامه کار و اتصال Clientهای دیگر را ساده‌تر می‌کند.

4-7 چالش‌ها و بدهی فنی

مهم‌ترین چالش، تصمیم‌نهایی درباره مرز داده و آماده‌سازی تولیدی است. تنظیمات توسعه‌ای باید از تنظیمات تولید جدا شود، رازها باید از محیط امن خوانده شوند، ورود اجتماعی به پیکربندی کامل نیاز دارد، و Caller در نسخه تولیدی باید با کنترل خطا، محدودیت زمانی و ثبت رویداد دقیق به سرویس مقصد متصل شود.

این چالش‌ها مانع نمایش پروژه نیستند، اما برای بهره‌برداری واقعی باید حل شوند. دواگانگی داده می‌تواند در آینده باعث اختلاف رفتار بین پنل مدیریت و مسیرهای API شود. تنظیمات توسعه‌ای نیز اگر بدون تغییر وارد محیط تولید شوند، خطر امنیتی ایجاد می‌کنند. همچنین پرداخت دستی و اجرای نمونه Caller باید در نسخه تولیدی با سرویس‌های واقعی، کنترل خطا، محدودیت زمانی و ثبت رویداد دقیق جایگزین یا تکمیل شوند.

5-7 بحث

پیاده‌سازی فعلی برای نمایش و آزمون قابلیت‌های اصلی بازارچه API مناسب است. با این حال، برای استفاده تولیدی باید مرز داده، محرمانگی تنظیمات، مدیریت سرویس واقعی MongoDB، آزمون‌های فرانت‌اند و مسیر پرداخت واقعی دقیق‌تر شود.

سطح بلوغ پروژه را می‌توان «نمونه کامل قابل اجرا برای محیط توسعه» دانست. قابلیت‌های اصلی محصول وجود دارند و آزمون بک‌اند از آن‌ها پشتیبانی می‌کند، اما همه نیازهای یک سامانه تولیدی هنوز کامل نشده‌اند؛ از جمله مدیریت رازها، اتصال‌های بیرونی واقعی، مشاهده‌پذیری، پایداری زیر بار و چرخه انتشار رسمی.

6-7 جمع‌بندی فصل

نتیجه پروژه یک پورتال نسبتاً کامل برای کشف، انتشار و مدیریت API است. بک‌اند در محدوده آزمون‌های فعلی وضعیت خوبی دارد، اما آماده‌سازی تولیدی هنوز کار می‌خواهد. همین موضوع مبنای پیشنهادهای فصل پایانی است.

فصل ۸

نتیجه‌گیری و کارهای آینده

در پایان گزارش، باید روشن باشد که پروژه چه ارزشی ایجاد کرده و چه فاصله‌ای تا یک محصول آماده بهره‌برداری دارد. این بخش فصل‌های قبل را تکرار نمی‌کند؛ بلکه جمع‌بندی می‌کند سامانه امروز در چه وضعیتی قرار دارد و ادامه کار بهتر است از کجا شروع شود.

1-8 جمع‌بندی کار انجام‌شده

IranAPI Hub یک پلتفرم اشتراک‌گذاری API با بک‌اند Django REST Framework و فرانت‌اند React است. سامانه کاتالوگ، مستندات، احراز هویت، Dashboard، مصرف، اشتراک، سازمان، انتشار، Caller و Studio را پوشش می‌دهد.

در طول گزارش، پروژه از دو زاویه بررسی شد: تجربه کاربر و ساختار فنی. فصل نیازمندی‌ها نقش‌ها و جریان‌ها را مشخص کرد. فصل معماری نشان داد این جریان‌ها چگونه بین فرانت‌اند، بک‌اند و داده تقسیم می‌شوند. فصل پیاده‌سازی رفتار بخش‌های اصلی را توضیح داد و فصل آزمون نشان داد بک‌اند با 49 آزمون موفق پشتیبانی می‌شود.

2-8 دستاوردهای اصلی

دستاوردهای اصلی پروژه شامل پشتیبانی از زبان فارسی، مسیرهای عمومی و خصوصی منظم، خروجی OpenAPI، تست‌های بک‌اند، ساختار Docker Compose و تفکیک نسبی بین لایه نمای API، سریال‌سازی، داده و رابط کاربری است.

دستآورد مهم دیگر، هماهنگی نسبی بین تجربه کاربر و معماری فنی است. کاربر کاتالوگ، مستندات، حساب و ابزارهای توسعه‌دهنده را می‌بیند و پشت آن، API، Serializer، لایه داده و آزمون قرار گرفته‌اند. همین رابطه باعث می‌شود پروژه فقط نمایشی نباشد و از نظر فنی هم قابل ادامه‌دادن باشد.

3-8 محدودیت‌های فعلی

محدودیت‌های فعلی شامل آماده نبودن تنظیمات تولید، باقی ماندن مسیرها و مدل‌های سازگار قدیمی، نبود اجرای تازه آزمون‌های فرانت‌اند در این گزارش، نبود اثبات اتصال واقعی Caller به سرویس خارجی و نیاز ورود اجتماعی به پیکربندی ارائه‌دهنده است. این محدودیت‌ها طبیعی‌اند، اما باید در برنامه ادامه کار جدی گرفته شوند. تا زمانی که تنظیمات تولید، رازها، پایگاه داده واقعی، پرداخت، اتصال بیرونی و آزمون مرورگر کامل نشده باشند، بهتر است پروژه به عنوان نسخه آموزشی یا نمونه قابل توسعه معرفی شود، نه محصول تولیدی آماده بهره‌برداری عمومی.

4-8 کارهای آینده

کارهای آینده پیشنهادی:

- انتقال رازها و تنظیمات حساس به متغیرهای محیطی و غیرفعال کردن DEBUG در تولید.
 - تصمیم‌گیری نهایی درباره مرز MongoDB و Django ORM و حذف دوگانگی غیرضروری.
 - اجرای منظم آزمون‌های Playwright و افزودن آن‌ها به فرایند CI.
 - اتصال Caller به سرویس‌های مقصد با کنترل امنیت، timeout و ثبت خطای واقعی.
 - تکمیل پرداخت واقعی، webhook و کنترل سهمیه مصرف.
 - افزودن مستندات نصب تولید، نسخه‌گذاری API و مانیتورینگ عملیاتی.
- ترتیب انجام این کارها بهتر است از ریسک‌های بنیادی شروع شود. ابتدا باید امنیت پیکربندی و رازها اصلاح شود، سپس تصمیم داده‌ای بین MongoDB واقعی و مدل‌های ORM نهایی شود. پس از آن، آزمون‌های مرورگر، پرداخت واقعی، مانیتورینگ و اتصال Caller می‌توانند با اطمینان بیشتری کامل شوند.

5-8 جمع‌بندی فصل

پروژه پایه فنی مناسبی برای یک بازارچه API دارد و آزمون‌های بک‌اند موجود آن با موفقیت اجرا شده‌اند. مسیر توسعه بعدی باید روی سخت‌سازی تولیدی، یکپارچه‌سازی داده و آزمون‌های انتهابه‌انتها متمرکز شود. با انجام این گام‌ها، سامانه می‌تواند از نمونه آموزشی کامل به محصولی پایدارتر و آماده‌تر برای استفاده واقعی تبدیل شود.

منابع

- [https:// .Python 3 Documentation .Python Software Foundation](https://docs.python.org/3/) [١]
[.docs.python.org/3/](https://docs.python.org/3/)
- [https://docs . Django Documentation .Django Software Foundation](https://docs.djangoproject.com/) [٢]
[.djangoproject.com/](https://docs.djangoproject.com/)
- [https://www. .Official Documentation .Django REST Framework](https://www.djangoproject.com/rest-framework/) [٣]
[.django-rest-framework.org/](https://www.djangoproject.com/rest-framework/)
- [.https://react.dev/ .React Documentation .Meta Open Source](https://react.dev/) [٤]
- [https://www.typescriptlang. .TypeScript Handbook .Microsoft](https://www.typescriptlang.org/docs/) [٥]
[.org/docs/](https://www.typescriptlang.org/docs/)
- [.https://vite.dev/guide/ .Vite Guide .Vite](https://vite.dev/guide/) [٦]
- [https://nodejs.org/ .Node.js Documentation .OpenJS Foundation](https://nodejs.org/docs/latest/api/) [٧]
[.docs/latest/api/](https://nodejs.org/docs/latest/api/)
- [https://docs.docker. .Docker Compose Documentation .Docker](https://docs.docker.com/compose/) [٨]
[.com/compose/](https://docs.docker.com/compose/)
- [https://www.mongodb.com/ .MongoDB Manual .MongoDB](https://www.mongodb.com/docs/manual/) [٩]
[.docs/manual/](https://www.mongodb.com/docs/manual/)
- [https://spec. .OpenAPI Specification .OpenAPI Initiative](https://spec.openapis.org/oas/latest.html) [١٠]
[.openapis.org/oas/latest.html](https://spec.openapis.org/oas/latest.html)
- [.https://docs.rapidapi.com/ .RapidAPI Docs .RapidAPI](https://docs.rapidapi.com/) [١١]
- [https://playwright.dev/ .Playwright Documentation .Microsoft](https://playwright.dev/docs/intro) [١٢]
[.docs/intro](https://playwright.dev/docs/intro)

واژه‌نامه

واژه فارسی	برابر انگلیسی
پلتفرم اشتراک گذاری API	API Marketplace
احراز هویت نشست	Session Authentication
Legacy Token	Legacy Token
Studio Flow	Studio Flow
مجوز دسترسی	Access Grant

پیوست الف

راهنمای اجرا و شواهد تکمیلی

1-الف اجرای محلی

اجرای محلی پروژه با Docker Compose انجام می‌شود:

```
1 | docker compose up --build
```

در این پیکربندی، فرانت‌اند روی `http://localhost:5173` و بک‌اند روی `http://localhost:8000/api` منتشر می‌شود.

این روش برای بررسی سریع کل سامانه مناسب است، زیرا هر دو بخش اصلی را هم‌زمان بالا می‌آورد. پس از اجرا، کاربر می‌تواند ابتدا مسیر سلامت بک‌اند را بررسی کند و سپس از طریق فرانت‌اند وارد جریان مرور کاتالوگ، ورود، داشبورد و ابزارهای توسعه‌دهنده شود. اگر یکی از سرویس‌ها اجرا نشود، باید لاگ همان سرویس در Docker Compose بررسی شود.

2-الف اجرای آزمون بک‌اند

دستور اجراشده در این گزارش:

```
1 | python manage.py test api
```

خروجی خلاصه: Ran 49 tests OK.

این دستور فقط آزمون‌های برنامه `api` را اجرا می‌کند و برای تکرار نتیجه فصل آزمون کافی است. در صورت تغییر کد بک‌اند، اجرای دوباره این دستور باید نخستین کنترل کیفی باشد. برای تغییرات فرانت‌اند، اجرای آزمون‌های مرورگر و ساخت نسخه فرانت‌اند نیز باید به این کنترل اضافه شود.

3-الف مسیرهای مهم

جدول الف-۱ مسیرهای مهم برای بررسی دستی سامانه

مسیر	کاربرد
/api/v1/system/health/	سلامت سامانه
/api/v1/schema/openapi.json	طرح OpenAPI
/api/v1/auth/login/	ورود نشست محور
/api/v1/catalog/apis/	فهرست و انتشار API
/api/v1/account/usage/	تاریخچه مصرف
/api/v1/account/studio/flows/	جریان های Studio